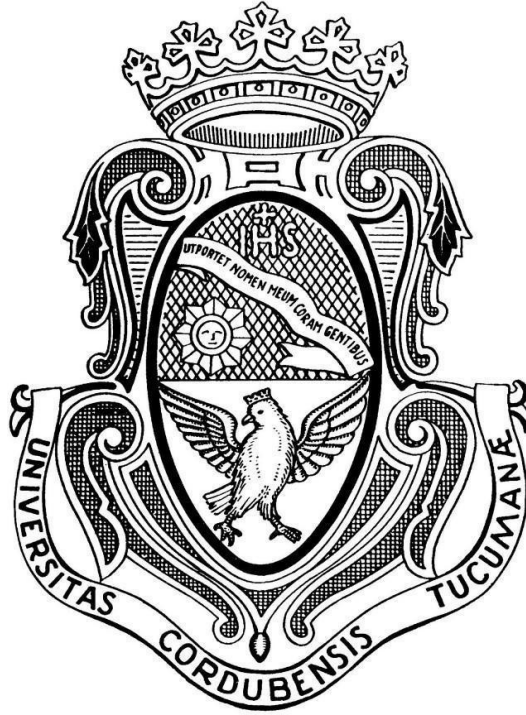


Universidad Nacional de Córdoba

Facultad de Ciencias Exactas, Físicas y Naturales



Sistemas de Computación

Trabajo Práctico N° 2 - Stack Frame

Alumno: Gastón Yomaha

Docentes: Javier Jorge
Miguel Solinas

21 de abril de 2026

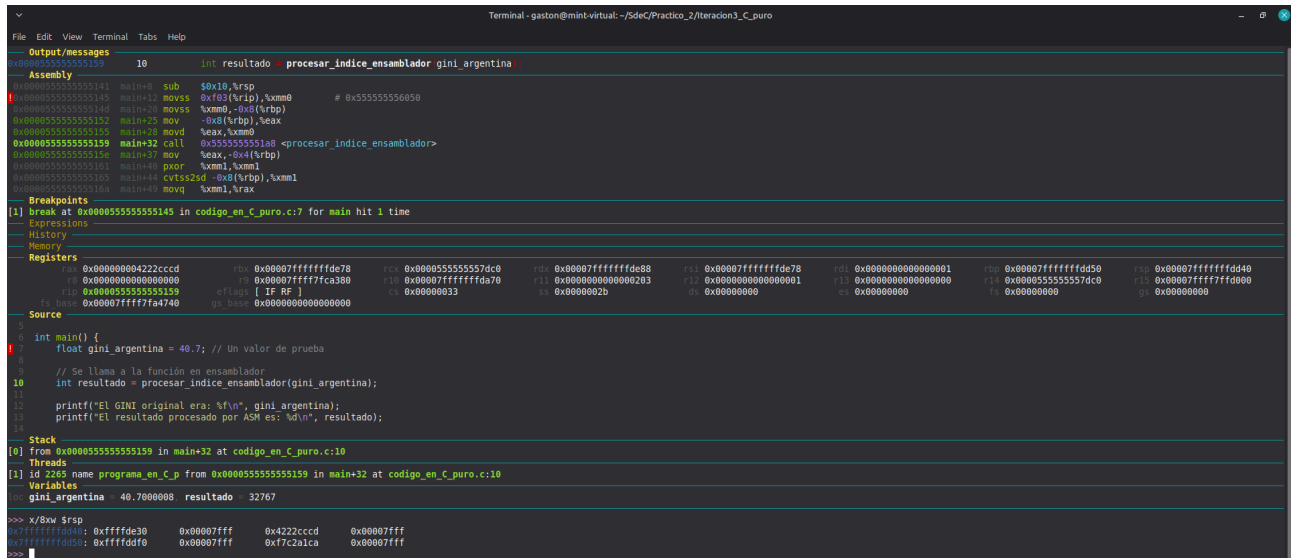
Índice

1	Estado Inicial	2
2	Stack Frame	2
3	Estado Final	9

1. Estado Inicial

Antes de ejecutar la instrucción `call`, se está ejecutando el `main` del archivo “programa_en_C_puro”.

Mediante el comando “`x/8xw $rsp`” se pueden ver 8 palabras de 32 bits a partir del stack pointer, lo cual es útil para ver como se modifican las variables de interés en la pila, tal como se puede apreciar en la Fig. 1:



```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/iteracion3_C_puro

Output/messages
10      int resultado = procesar_indice_ensamblador(gini_argentina);

Assembly
0x0000555555551159: sub $0x10,%rsp
0x0000555555551161: movss 0xf03(%rip),%xmm0 # 0x55555556050
0x0000555555551164: movss %xmm0,-0x8(%rbp)
0x0000555555551167: mov -0x8(%rbp),%eax
0x0000555555551169: movd %eax,%xmm0
0x000055555555116b: call 0x5555555511a8 <procesar_indice_ensamblador>
0x000055555555116e: mov %eax,-0x4(%rbp)
0x0000555555551170: pldr %xmm1,%xmm1
0x0000555555551172: cvtsd2si -0x8(%rbp),%xmm1
0x0000555555551174: movq %xmm1,%rax

Breakpoints
[1] break at 0x0000555555551164 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x00000000422cccd rdx 0x00007fffffffd78 rdi 0x0000555555557dc0 rsi 0x00007fffffffd78 r10 0x0000000000000001 r11 0x00007fffffffd50 r12 0x00007ffffffdd40
r13 0x0000000000000000 r14 0x00007fffffffc30 r15 0x00007ffffffda70 r16 0x0000000000000203 r17 0x0000000000000001 r18 0x0000000000000000 r19 0x0000000000000000
r20 0x0000000000000000 r21 0x0000000000000000 r22 0x0000000000000000 r23 0x0000000000000000 r24 0x0000000000000000 r25 0x0000000000000000 r26 0x0000000000000000
r27 0x0000000000000000 r28 0x0000000000000000 r29 0x0000000000000000 r30 0x0000000000000000 r31 0x0000000000000000
rsi_base 0x00007fffffffa4740 gs_base 0x0000000000000000

Source
1 int main() {
2     float gini_argentina = 40.7; // Un valor de prueba
3
4     // Se llama a la función en ensamblador
5     int resultado = procesar_indice_ensamblador(gini_argentina);
6
7     printf("El GINI original era: %f\n", gini_argentina);
8     printf("El resultado procesado por ASM es: %d\n", resultado);
9 }

Stack
[0] from 0x0000555555551159 in main+32 at codigo_en_C_puro.c:10
Threads
[1] id 2248 name programa_en_C_p from 0x0000555555551159 in main+32 at codigo_en_C_puro.c:10
Variables
gini_argentina 40.7000008 resultado 32767

>>> x/8xw $rsp
0x7fffffffd40: 0xffffde30 0x00007fff 0x422cccd 0x00007fff
0x7ffffffd30: 0xffffdd00 0x00007fff 0xf7c2a1ca 0x00007fff
    
```

Figura 1: Antes de entrar al código en ensamblador.

El registro del stack pointer (`$rsp`) apunta a la dirección de memoria `0x00007ffffffdd40`, mientras que el registro del base pointer (`$rbp`) apunta a la dirección de memoria `0x00007ffffffdd50`. Se puede comprobar que en la dirección `0x00007ffffffdd48`, que es 8 bytes menor a la del `$rbp`, se identifica el valor `0x422cccd`, que corresponde al valor 40.7, es decir la variable de prueba “`gini_argentina`”. Este número es un float, por lo que se guarda de una forma particular, donde el primer bit (de izquierda a derecha) corresponde al signo (en este caso es un cero, así que el número es positivo). Los bits 2 al 9 son de exponente y los 23 bits restantes corresponden a la mantisa.

2. Stack Frame

Al ejecutar la instrucción `call`, se hace un push de la dirección de retorno hacia `main` en el stack. Entonces el `$rsp` desciende 8 bytes, pasando de `0x00007ffffffdd40` a `0x00007ffffffdd38`, como se puede ver en la Fig. 2.

Lo primero que se hace una vez dentro del código en assembler es guardar el valor original de `$rbp` mediante la instrucción `pushq`, como se ve en la Fig. 3. Esto provoca que `$rsp` disminuya de nuevo, quedando en `0x00007ffffffdd30`. En este punto se puede ver en la parte inferior del gdb dashboard que en la posición de `$rsp` está guardado el valor de `$rbp`, utilizando Little Endian.

Luego se copia el valor del `$rsp` en el `$rbp` para poder usarlo como referencia mediante la instrucción `movq`, como se ve en la Fig. 4.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
procesar_indice_ensamblador () at float_a_entero.s:6
6      pushq   %rbp           # Se guarda el base pointer de la función que llama

-- Assembly
0x0000555555551a8 ? push %rbp
0x0000555555551a9 ? mov %rsp,%rbp
0x0000555555551aa ? sub $0x10,%rsp
0x0000555555551ab ? movss %xmm0,-0x4(%rbp)
0x0000555555551ac ? movss -0x4(%rbp),%xmm1
0x0000555555551ad ? cvtsq2si %xmm1,%eax
0x0000555555551ae ? add $0x1,%eax
0x0000555555551af ? mov %eax,-0x8(%rbp)
0x0000555555551b0 ? mov -0x8(%rbp),%eax
0x0000555555551b1 ? mov %rbp,%rsp

-- Breakpoints
[1] break at 0x0000555555551a5 in codigo_en_c_puro.c:7 for main hit 1 time

-- Expressions
-- History
-- Memory
-- Registers
rax 0x00000000422cccd rbp 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd50 r13 0x00007fffffffd35
rip 0x0000555555551a8 eflags 0x0000000000000000 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r15 0x0000000000000000 r14 0x0000555555557dc0 r16 0x00007fffffffd00
fs base 0x00007ffffffa4740 gs base 0x0000000000000000

-- Source
1 .global procesar_indice_ensamblador
2 .text
3
4 procesar_indice_ensamblador:
5     # --- PROLOGO DEL STACK FRAME ---
6     pushq   %rbp           # Se guarda el base pointer de la función que llama
7     movq    %rsp, %rbp     # Se establece un base pointer propio
8
9     # --- RESERVAR ESPACIO EN EL STACK ---
10    # Se Restan 16 bytes al puntero del stack (%rsp) para crear espacio
11
12 -- Stack
[0] from 0x0000555555551a8 in procesar_indice_ensamblador at float_a_entero.s:6
[1] from 0x00005555555515e in main+17 at codigo_en_c_puro.c:10
-- Threads
[1] id 2268 name programa_en_c_p from 0x0000555555551a8 in procesar_indice_ensamblador at float_a_entero.s:6
-- Variables
-->>> x/Box $rsp
0x7fffffffd50: 0x5555515e 0x00005555 0xffffde30 0x00007fff
0x7ffffffd040: 0x422cccd 0x00007fff 0xffffdd9 0x00007fff
-->>>

```

Figura 2: call.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
7      movq    %rsp, %rbp     # Se establece un base pointer propio

-- Assembly
0x0000555555551a9 ? mov %rsp,%rbp
0x0000555555551aa ? sub $0x10,%rsp
0x0000555555551ab ? movss %xmm0,-0x4(%rbp)
0x0000555555551ac ? movss -0x4(%rbp),%xmm1
0x0000555555551ad ? cvtsq2si %xmm1,%eax
0x0000555555551ae ? add $0x1,%eax
0x0000555555551af ? mov %eax,-0x8(%rbp)
0x0000555555551b0 ? mov -0x8(%rbp),%eax
0x0000555555551b1 ? mov %rbp,%rsp
0x0000555555551b2 ? pop %rbp

-- Breakpoints
[1] break at 0x0000555555551a5 in codigo_en_c_puro.c:7 for main hit 1 time

-- Expressions
-- History
-- Memory
-- Registers
rax 0x00000000422cccd rbp 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd50 r13 0x00007fffffffd35
rip 0x0000555555551a9 eflags 0x0000000000000000 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r15 0x0000000000000000 r14 0x0000555555557dc0 r16 0x00007ffffffd00
fs base 0x00007ffffffa4740 gs base 0x0000000000000000

-- Source
1 .global procesar_indice_ensamblador
2 .text
3
4 procesar_indice_ensamblador:
5     # --- PROLOGO DEL STACK FRAME ---
6     pushq   %rbp           # Se guarda el base pointer de la función que llama
7     movq    %rsp, %rbp     # Se establece un base pointer propio
8
9     # --- RESERVAR ESPACIO EN EL STACK ---
10    # Se Restan 16 bytes al puntero del stack (%rsp) para crear espacio
11    # para las "variables locales" (parámetro y resultado).
12
13 -- Stack
[0] from 0x0000555555551a9 in procesar_indice_ensamblador at float_a_entero.s:7
[1] from 0x00005555555515e in main+17 at codigo_en_c_puro.c:10
-- Threads
[1] id 2268 name programa_en_c_p from 0x0000555555551a9 in procesar_indice_ensamblador at float_a_entero.s:7
-- Variables
-->>> x/Box $rsp
0x7fffffffd50: 0xffffdd50 0x00007fff 0x5555515e 0x00005555
0x7ffffffd040: 0xffffde30 0x00007fff 0x422cccd 0x00007fff
-->>>

```

Figura 3: Pushq %rbp.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/iteracion3_C_puro

Output/messages
13 subq $16, %rsp

Assembly
0x00005555555511ac 7 sub $0x10,%rsp
0x00005555555511ad 7 movss %xmm0,-0x4(%rbp)
0x00005555555511ae 7 movss -0x4(%rbp),%xmm1
0x00005555555511af 7 cvtsq2si %xmm1,%eax
0x00005555555511b0 7 add $0x1,%eax
0x00005555555511b1 7 mov %eax,-0x8(%rbp)
0x00005555555511b2 7 mov -0x8(%rbp),%eax
0x00005555555511b3 7 mov %rbp,%rsp
0x00005555555511b4 7 pop %rbp
0x00005555555511b5 7 ret

Breakpoints
[1] break at 0x0000555555551145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Registers
rax 0x00000000422cccd rbx 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 r8 0x00007ffffffdd30
rsi 0x0000000000000000 r9 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007ffffffdd000
r16 base 0x00007ffffffa4740 r17 base 0x0000000000000000 r18 0x00000003 r19 0x0000002b r20 0x00000000 r21 0x00000000 r22 0x00000000 r23 0x00000000 r24 0x00000000

Source
8 # --- RESERVAR ESPACIO EN EL STACK ---
9 # Se Restan 16 bytes al puntero del stack (%rsp) para crear espacio
10 # para las "variables locales" (parámetro y resultado).
11 # Se restan 16 y no menos para mantener la alineación de memoria obligatoria. (según convención de llamadas)
12 subq $16, %rsp
13
14 # --- USAR EL STACK PARA EL PARÁMETRO ---
15 # Se guarda el float (que llegó en %xmm0) en la memoria del stack (a -4 bytes del base pointer)
16 movss %xmm0, -4(%rbp)
17
Stack
[0] from 0x00005555555511ac in procesar_indice_ensamblador at float_a_entrero.s:13
[1] from 0x00005555555511b5 in main+17 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x00005555555511ac in procesar_indice_ensamblador at float_a_entrero.s:13
Variables
>>> x/Rwx $rsp
0x7fffffffd030: 0xffffdd50 0x00007fff 0x5555515e 0x00005555
0x7fffffffd030: 0xffffdd50 0x00007fff 0x422cccd 0x00007fff
>>>
    
```

 Figura 4: `movq %rsp, %rbp`.

Ahora se hace espacio para variables locales mediante la instrucción `subq`, donde se restan 16 bytes al `$rsp`, como se ve en la Fig. 5.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/iteracion3_C_puro

Output/messages
17 movss %xmm0, -4(%rbp)

Assembly
0x00005555555511b0 7 movss %xmm0,-0x4(%rbp)
0x00005555555511b1 7 movss -0x4(%rbp),%xmm1
0x00005555555511b2 7 cvtsq2si %xmm1,%eax
0x00005555555511b3 7 add $0x1,%eax
0x00005555555511b4 7 mov %eax,-0x8(%rbp)
0x00005555555511b5 7 mov -0x8(%rbp),%eax
0x00005555555511b6 7 mov %rbp,%rsp
0x00005555555511b7 7 pop %rbp
0x00005555555511b8 7 ret
0x00005555555511b9 7 endbr64

Breakpoints
[1] break at 0x0000555555551145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Registers
rax 0x00000000422cccd rbx 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007ffffffdd30 r8 0x00007ffffffdd30
rsi 0x0000000000000000 r9 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007ffffffdd000
r16 base 0x00007ffffffa4740 r17 base 0x0000000000000000 r18 0x00000003 r19 0x0000002b r20 0x00000000 r21 0x00000000 r22 0x00000000 r23 0x00000000 r24 0x00000000

Source
12 # Se restan 16 y no menos para mantener la alineación de memoria obligatoria. (según convención de llamadas)
13 subq $16, %rsp
14
15 # --- USAR EL STACK PARA EL PARÁMETRO ---
16 # Se guarda el float (que llegó en %xmm0) en la memoria del stack (a -4 bytes del base pointer)
17 movss %xmm0, -4(%rbp)
18
19 # --- LÓGICA DE CONVERSIÓN ---
20 # Se lee el float desde la pila y se pone en otro registro para procesarlo.
21 movss -4(%rbp), %xmm1
22
Stack
[0] from 0x00005555555511b0 in procesar_indice_ensamblador at float_a_entrero.s:17
[1] from 0x00005555555511b6 in main+17 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x00005555555511b0 in procesar_indice_ensamblador at float_a_entrero.s:17
Variables
>>> x/Rwx $rsp
0x7fffffffd030: 0x00000000 0x00000000 0x00000000 0x00000000
0x7fffffffd030: 0xffffdd50 0x00007fff 0x5555515e 0x00005555
>>>
    
```

 Figura 5: `subq $16, %rsp`.

Luego, mediante `movss`, se guarda el float (que llegó en `%xmm0` por convención de llamadas) como variable local, tal como se ve en la Fig. 6.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
21 movss -0x4(%rbp), %xmm1

Assembly
0x0000555555551b5 ? movss -0x4(%rbp), %xmm1
0x0000555555551b6 ? cvtss2si %xmm1, %eax
0x0000555555551b7 ? add $0x1, %eax
0x0000555555551c1 ? mov %eax, -0x8(%rbp)
0x0000555555551c4 ? mov -0x8(%rbp), %eax
0x0000555555551c7 ? mov %rbp, %rsp
0x0000555555551ca ? pop %rbp
0x0000555555551cb ? ret
0x0000555555551cc ? endbr64
0x0000555555551cd ? sub $0x8, %rsp

Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x00000000422cccd rbx 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 rsp 0x00007fffffffd20
rsi 0x0000000000000000 r9 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007fffffffd00
eflags [ IF RF ] cs 0x000000033 ds 0x00000002b fs 0x000000000 gs 0x000000000

Source
16 # Se guarda el float (que llegó en %xmm0) en la memoria del stack (a -4 bytes del base pointer)
17 movss -4(%rbp), %xmm1
18
19 # --- LÓGICA DE CONVERSIÓN ---
20 # Se lee el float desde la pila y se pone en otro registro para procesarlo.
21 movss -4(%rbp), %xmm1
22
23 # Convertir Scalar Single-Precision FP to Integer:
24 # CVTSS2SI redondea automáticamente al entero más cercano:
25 cvtss2si %xmm1, %eax

Stack
[0] from 0x0000555555551b5 in procesar_indice_ensamblador at float_a_entero.s:21
[1] from 0x00005555555515e in main+9 at codigo_en_C_puro.c:10

Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551b5 in procesar_indice_ensamblador at float_a_entero.s:21

Variables
>>> x/Rwx $rsp
0x7fffffffd20: 0x00000000 0x00000000 0x00000000 0x422cccd
0x7ffffffd30: 0xffffd50 0x00007fff 0x5555515e 0x00005555
>>>
    
```

 Figura 6: `movss %xmm0, -4(%rbp)`.

Ahora se lee el valor guardado previamente y se lo guarda en otro registro (`%xmm1`), según la Fig. 7.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
25 cvtss2si %xmm1, %eax

Assembly
0x0000555555551ba ? cvtss2si %xmm1, %eax
0x0000555555551bb ? add $0x1, %eax
0x0000555555551c1 ? mov %eax, -0x8(%rbp)
0x0000555555551c4 ? mov -0x8(%rbp), %eax
0x0000555555551c7 ? mov %rbp, %rsp
0x0000555555551ca ? pop %rbp
0x0000555555551cb ? ret
0x0000555555551cc ? endbr64
0x0000555555551cd ? sub $0x8, %rsp
0x0000555555551ce ? add $0x8, %rsp

Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x00000000422cccd rbx 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 rsp 0x00007fffffffd20
rsi 0x0000000000000000 r9 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007fffffffd00
eflags [ IF RF ] cs 0x000000033 ds 0x00000002b fs 0x000000000 gs 0x000000000

Source
18 # Se lee el float desde la pila y se pone en otro registro para procesarlo.
19 movss -4(%rbp), %xmm1
20
21 # Convertir Scalar Single-Precision FP to Integer:
22 # CVTSS2SI redondea automáticamente al entero más cercano:
23 cvtss2si %xmm1, %eax
24
25 # --- SUMAR UNO ---
26 # Se le suma 1 al registro %eax que ahora contiene el entero
27 addl $1, %eax

Stack
[0] from 0x0000555555551ba in procesar_indice_ensamblador at float_a_entero.s:25
[1] from 0x00005555555515e in main+9 at codigo_en_C_puro.c:10

Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551ba in procesar_indice_ensamblador at float_a_entero.s:25

Variables
>>> x/Rwx $rsp
0x7fffffffd20: 0x00000000 0x00000000 0x00000000 0x422cccd
0x7ffffffd30: 0xffffd50 0x00007fff 0x5555515e 0x00005555
>>>
    
```

 Figura 7: `movss -4(%rbp), %xmm1`.

Ahora se utiliza la instrucción “`cvtss2si`” para convertir el número desde float a entero, incluyendo el redondeo, como se ve en la Fig 8. Por convención de llamadas, el resultado se guarda en “`eax`”.

Ahora lo único que queda por hacer es sumarle 1 al número entero mediante la instrucción “`addl`”, como se ve en la Fig. 9.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/Iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
29 addl $1,%eax

Assembly
0x0000555555551be 7 add $0x1,%eax
0x0000555555551c1 7 mov %eax,-0x8(%rbp)
0x0000555555551c4 7 mov -0x8(%rbp),%eax
0x0000555555551c7 7 mov %rbp,%rsp
0x0000555555551ca 7 pop %rbp
0x0000555555551cb 7 ret
0x0000555555551cc 7 endbr64
0x0000555555551d0 7 sub $0x8,%rsp
0x0000555555551d4 7 add $0x8,%rsp
0x0000555555551d8 7 ret

Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x0000000000000029 rdx 0x00007fffffffd78 rsi 0x0000555555557dc0 rdi 0x00007fffffffd88 r10 0x00007fffffffd78 r11 0x0000000000000001 r12 0x00007fffffffd30 r13 0x00007fffffffd20
rbx 0x0000000000000000 r9 0x00007ffffffca30 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000000 r13 0x0000555555557dc0 r14 0x00007fffffffd000
rcx 0x0000555555551be rflags 0x0000000000000000 rbp 0x0000000000000000 rsi 0x0000000000000000 rdi 0x0000000000000000 r14 0x0000000000000000 r15 0x0000000000000000
rsi base 0x00007ffffffa4740 gs base 0x0000000000000000

Source
24 # Cvtss2si redondea automáticamente al entero más cercano:
25 cvtss2si %xmm1,%eax
26
27 # --- SUMAR UNO ---
28 # Se le suma 1 al registro %eax que ahora contiene el entero
29 addl $1,%eax
30
31 # --- USAR EL STACK PARA DEVOLVER EL RESULTADO ---
32 # Se guarda el entero resultante en el stack (a -8 bytes de la base pointer)
33 movl %eax,-8(%rbp)
34 movl -8(%rbp),%eax
35
Stack
[0] from 0x0000555555551be in procesar_indice_ensamblador at float_a_entero.s:29
[1] from 0x0000555555551be in main+37 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551be in procesar_indice_ensamblador at float_a_entero.s:29
Variables
>>> x/Rwx $rsp
0x7fffffffd020: 0x00000000 0x00000000 0x00000000 0x4222cccc
0x7fffffffd030: 0xffffdd50 0x00007fff 0x5555551e 0x00005555
>>>
    
```

 Figura 8: `cvtss2si %xmm1,%eax`.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/Iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
33 movl %eax,-8(%rbp)

Assembly
0x0000555555551c1 7 mov %eax,-0x8(%rbp)
0x0000555555551c4 7 mov -0x8(%rbp),%eax
0x0000555555551c7 7 mov %rbp,%rsp
0x0000555555551ca 7 pop %rbp
0x0000555555551cb 7 ret
0x0000555555551cc 7 endbr64
0x0000555555551d0 7 sub $0x8,%rsp
0x0000555555551d4 7 add $0x8,%rsp
0x0000555555551d8 7 ret
0x0000555555551db 7 add %al,%rax

Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x000000000000002a rdx 0x00007fffffffd78 rsi 0x0000555555557dc0 rdi 0x00007fffffffd88 r10 0x00007fffffffd78 r11 0x0000000000000001 r12 0x00007fffffffd30 r13 0x00007fffffffd20
rbx 0x0000000000000000 r9 0x00007ffffffca30 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000000 r13 0x0000555555557dc0 r14 0x00007fffffffd000
rcx 0x0000555555551c1 rflags 0x0000000000000000 rbp 0x0000000000000000 rsi 0x0000000000000000 rdi 0x0000000000000000 r14 0x0000000000000000 r15 0x0000000000000000
rsi base 0x00007ffffffa4740 gs base 0x0000000000000000

Source
28 # Se le suma 1 al registro %eax que ahora contiene el entero
29 addl $1,%eax
30
31 # --- USAR EL STACK PARA DEVOLVER EL RESULTADO ---
32 # Se guarda el entero resultante en el stack (a -8 bytes de la base pointer)
33 movl %eax,-8(%rbp)
34 movl -8(%rbp),%eax
35
36 # --- EPILOGO DEL STACK FRAME ---
37 # Primero se restaura el puntero de la pila para "borrar" las variables locales

Stack
[0] from 0x0000555555551c1 in procesar_indice_ensamblador at float_a_entero.s:33
[1] from 0x0000555555551be in main+37 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551c1 in procesar_indice_ensamblador at float_a_entero.s:33
Variables
>>> x/Rwx $rsp
0x7fffffffd020: 0x00000000 0x00000000 0x00000000 0x4222cccc
0x7fffffffd030: 0xffffdd50 0x00007fff 0x5555551e 0x00005555
>>>
    
```

 Figura 9: `addl $1,%eax`.

Ahora se utiliza el stack para devolver el resultado, según se ve en las Fig. 10 y 11. Cabe destacar que se hace de esta forma solamente para demostrar el uso de la pila y no porque tenga una utilidad en particular.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/Iteracion3_C_puro

File Edit View Terminal Tabs Help

34 Output/messages
Assembly
0x0000555555551c4 ? mov -0x8(%rbp),%eax
0x0000555555551c7 ? mov %rbp,%rsp
0x0000555555551ca ? pop %rbp
0x0000555555551cb ? ret
0x0000555555551cc ? endbr64
0x000055555555108 ? sub $0x8,%rsp
0x000055555555104 ? add $0x8,%rsp
0x000055555555108 ? ret
0x000055555555109 ? add %al,(%rax)
0x00005555555510b ? add %al,(%rax)

Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Registers
rax 0x000000000000002a rbx 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 rsp 0x00007fffffffd20
rip 0x0000555555551c4 r19 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007ffffffd000
fs_base 0x00007ffffffa4740 gs_base 0x0000000000000000 cr0 0x00000033 cr2 0x0000002b cr3 0x00000000 cr4 0x00000000 cr8 0x00000000

Source
34 addl $1,%eax
35
36 # --- USAR EL STACK PARA DEVOLVER EL RESULTADO ---
37 # Se guarda el entero resultante en el stack (a -8 bytes del base pointer)
38 movl %eax,-8(%rbp)
39 movl -8(%rbp),%eax
40
41 # --- EPILOGO DEL STACK FRAME ---
42 # Primero se restaura el puntero de la pila para "borrar" las variables locales
43 movq %rbp,%rsp
44
Stack
[0] from 0x0000555555551c4 in procesar_indice_ensamblador at float_a_entero.s:34
[1] from 0x000055555555110 in main+19 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551c4 in procesar_indice_ensamblador at float_a_entero.s:34
Variables
>>> x/Bxw $rsp
0x7fffffffd20: 0x00000000 0x00000000 0x0000002a 0x4222cccc
0x7ffffffd30: 0xffffdd50 0x00007fff 0x5555515e 0x00005555
>>>
    
```

 Figura 10: `movl %eax, -8(%rbp)`.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/Iteracion3_C_puro

File Edit View Terminal Tabs Help

38 Output/messages
Assembly
0x0000555555551c7 ? mov %rbp,%rsp
0x0000555555551ca ? pop %rbp
0x0000555555551cb ? ret
0x0000555555551cc ? endbr64
0x000055555555108 ? sub $0x8,%rsp
0x000055555555104 ? add $0x8,%rsp
0x000055555555108 ? ret
0x000055555555109 ? add %al,(%rax)
0x00005555555510b ? add %al,(%rax)
0x00005555555510d ? add %al,(%rax)

Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Registers
rax 0x000000000000002a rbx 0x00007fffffffd78 rcx 0x0000555555557dc0 rdx 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 rsp 0x00007fffffffd20
rip 0x0000555555551c7 r19 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007ffffffd000
fs_base 0x00007ffffffa4740 gs_base 0x0000000000000000 cr0 0x00000033 cr2 0x0000002b cr3 0x00000000 cr4 0x00000000 cr8 0x00000000

Source
38 movl %eax,-8(%rbp)
39 movl -8(%rbp),%eax
40
41 # --- EPILOGO DEL STACK FRAME ---
42 # Primero se restaura el puntero de la pila para "borrar" las variables locales
43 movq %rbp,%rsp
44 # Se deshace el stack frame antes de volver
45 popq %rbp
46
47 # Se retorna a la función que llama. El resultado viaja en %eax.

Stack
[0] from 0x0000555555551c7 in procesar_indice_ensamblador at float_a_entero.s:38
[1] from 0x000055555555110 in main+19 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551c7 in procesar_indice_ensamblador at float_a_entero.s:38
Variables
>>> x/Bxw $rsp
0x7fffffffd20: 0x00000000 0x00000000 0x0000002a 0x4222cccc
0x7ffffffd30: 0xffffdd50 0x00007fff 0x5555515e 0x00005555
>>>
    
```

 Figura 11: `movl -8(%rbp), %eax`.

Al finalizar los cálculos, es necesario limpiar el stack frame antes de volver hacia el main, por lo que primero se vuelve el `$rsp` al principio mediante `movq`, como se ve en Fig. 12.

Luego se recupera el valor original de `%rbp`, según Fig. 13.


```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
40      popq    %rbp
Assembly
0x0000555555551ca 7 pop    %rbp
0x0000555555551cb 7 ret
0x0000555555551cc 7 endbr64
0x0000555555551cd 7 sub    $0x8,%rsp
0x0000555555551ce 7 add    $0x8,%rsp
0x0000555555551cf 7 ret
0x0000555555551d0 7 add    %al,(%rax)
0x0000555555551d1 7 add    %al,(%rax)
0x0000555555551d2 7 add    %al,(%rax)
0x0000555555551d3 7 add    %al,(%rax)
Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time
Expressions
History
Memory
Registers
rax 0x000000000000002a rdx 0x00007fffffffd78 rcx 0x0000555555557dc0 r8 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 r10 0x00007fffffffd30
rbx 0x0000000000000000 r9 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007ffffffd000
rip 0x0000555555551ca eflags 0x0000000000000000 cs 0x000000033 ss 0x00000002b ds 0x000000000 fs 0x000000000 gs 0x000000000
fs base 0x00007ffffffa4740 gs base 0x0000000000000000
Source
35      # --- EPILOGO DEL STACK FRAME ---
36      # Primero se restaura el puntero de la pila para "borrar" las variables locales
37      movq    %rbp, %rsp
38      # Se desarma el stack frame antes de volver
39      popq    %rbp
40      # Se retorna a la función que llama. El resultado viaja en %eax.
41      ret
42
Stack
[0] from 0x0000555555551ca in procesar_indice_ensamblador at float_a_entrero.s:40
[1] from 0x00005555555515e in main+37 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551ca in procesar_indice_ensamblador at float_a_entrero.s:40
Variables
>>> x/8wx $rsp
0x7fffffffd30: 0xfffffd50 0x00007fff 0x5555515e 0x00005555
0x7ffffffd040: 0xfffffd30 0x00007fff 0x4222cccd 0x00007fff
>>>
    
```

Figura 12: movq %rbp, %rsp.

```

Terminal - gaston@mint-virtual: ~/5deC/Practico_2/iteracion3_C_puro

File Edit View Terminal Tabs Help

Output/messages
43      ret
Assembly
0x0000555555551cb 7 ret
0x0000555555551cc 7 endbr64
0x0000555555551cd 7 sub    $0x8,%rsp
0x0000555555551ce 7 add    $0x8,%rsp
0x0000555555551cf 7 ret
0x0000555555551d0 7 add    %al,(%rax)
0x0000555555551d1 7 add    %al,(%rax)
0x0000555555551d2 7 add    %al,(%rax)
0x0000555555551d3 7 add    %al,(%rax)
Breakpoints
[1] break at 0x000055555555145 in codigo_en_C_puro.c:7 for main hit 1 time
Expressions
History
Memory
Registers
rax 0x000000000000002a rdx 0x00007fffffffd78 rcx 0x0000555555557dc0 r8 0x00007fffffffd88 rsi 0x00007fffffffd78 rdi 0x0000000000000001 rbp 0x00007fffffffd30 r10 0x00007fffffffd30
rbx 0x0000000000000000 r9 0x00007ffffffca380 r10 0x00007ffffffda70 r11 0x0000000000000203 r12 0x0000000000000001 r13 0x0000000000000000 r14 0x0000555555557dc0 r15 0x00007ffffffd000
rip 0x0000555555551cb eflags 0x0000000000000000 cs 0x000000033 ss 0x00000002b ds 0x000000000 fs 0x000000000 gs 0x000000000
fs base 0x00007ffffffa4740 gs base 0x0000000000000000
Source
38      movq    %rbp, %rsp
39      # Se desarma el stack frame antes de volver
40      popq    %rbp
41
42      # Se retorna a la función que llama. El resultado viaja en %eax.
43      ret
Stack
[0] from 0x0000555555551cb in procesar_indice_ensamblador at float_a_entrero.s:43
[1] from 0x00005555555515e in main+37 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000555555551cb in procesar_indice_ensamblador at float_a_entrero.s:43
Variables
>>> x/8wx $rsp
0x7ffffffd030: 0x5555515e 0x00005555 0xfffffd30 0x00007fff
0x7ffffffd040: 0x4222cccd 0x00007fff 0xfffffd00 0x00007fff
>>>
    
```

Figura 13: popq %rbp.

Por último, se retorna hacia el main mediante la instrucción ret, como muestra la Fig. 14

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/iteracion3_C_puro

-- Output/messages
0x0000000000000000 in main () at codigo_en_C_puro.c:10
10 int resultado = procesar_indice_ensamblador(gini_argentina);

Assembly
0x0000000000000000 main+12 movss %xmm0, -0x8(%rbp) # 0x555555556000
0x0000000000000000 main+20 movss %xmm0, -0x8(%rbp), %eax
0x0000000000000000 main+25 mov -0x8(%rbp), %eax
0x0000000000000000 main+28 movd %eax, %xmm0
0x0000000000000000 main+32 call 0x555555551a08 <procesar_indice_ensamblador>
0x0000000000000000 main+37 mov %eax, -0x4(%rbp)
0x0000000000000000 main+40 pxor %xmm1, %xmm1
0x0000000000000000 main+44 cvtsd2q -0x8(%rbp), %xmm1
0x0000000000000000 main+48 movq %xmm1, %rax
0x0000000000000000 main+54 movq %rax, %xmm0

Breakpoints
[1] break at 0x0000000000000000 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x0000000000000002a rdx 0x0000000000000000 rdi 0x0000000000000000 rsi 0x0000000000000000 rbp 0x0000000000000000 r10 0x0000000000000000 r11 0x0000000000000000 r12 0x0000000000000000 r13 0x0000000000000000 r14 0x0000000000000000 r15 0x0000000000000000
rip 0x0000000000000000 eflags 0x0000000000000000 cs base 0x0000000000000000 gs base 0x0000000000000000

Source
7 int main() {
8     float gini_argentina = 40.7; // Un valor de prueba
9
10    // Se llama a la función en ensamblador
11    int resultado = procesar_indice_ensamblador(gini_argentina);
12    printf("El GINI original era: %f\n", gini_argentina);
13    printf("El resultado procesado por ASM es: %d\n", resultado);
14
Stack
[0] from 0x0000000000000000 in main+37 at codigo_en_C_puro.c:10
Threads
[1] id 2265 name programa_en_C_p from 0x0000000000000000 in main+37 at codigo_en_C_puro.c:10
Variables
gini_argentina 40.7000000 resultado 32767

>>> x/8w $rsp
0x7fffffff0d40: 0xffffde30 0x00007fff 0x4222cccd 0x00007fff
0x7fffffff0d50: 0xffffd0f0 0x00007fff 0xf7c2a1ca 0x00007fff
>>>

```

Figura 14: ret

3. Estado Final

Al retornar de la función de ensamblador, lo único que queda por hacer es cargar el resultado en la variable correspondiente, como se ve en la Fig. 15.

```

Terminal - gaston@mint-virtual: ~/SdeC/Practico_2/iteracion3_C_puro

-- Output/messages
12 printf("El GINI original era: %f\n", gini_argentina);

Assembly
0x0000000000000000 main+12 movss %xmm0, -0x8(%rbp)
0x0000000000000000 main+20 movss %xmm0, -0x8(%rbp), %eax
0x0000000000000000 main+25 mov -0x8(%rbp), %eax
0x0000000000000000 main+28 movd %eax, %xmm0
0x0000000000000000 main+32 call 0x555555551a08 <procesar_indice_ensamblador>
0x0000000000000000 main+37 mov %eax, -0x4(%rbp)
0x0000000000000000 main+40 pxor %xmm1, %xmm1
0x0000000000000000 main+44 cvtsd2q -0x8(%rbp), %xmm1
0x0000000000000000 main+48 movq %xmm1, %rax
0x0000000000000000 main+54 movq %rax, %xmm0
0x0000000000000000 main+58 lea 0x0000000000000000(%rip), %rax # 0x555555556000

Breakpoints
[1] break at 0x0000000000000000 in codigo_en_C_puro.c:7 for main hit 1 time

Expressions
History
Memory
Registers
rax 0x0000000000000002a rdx 0x0000000000000000 rdi 0x0000000000000000 rsi 0x0000000000000000 rbp 0x0000000000000000 r10 0x0000000000000000 r11 0x0000000000000000 r12 0x0000000000000000 r13 0x0000000000000000 r14 0x0000000000000000 r15 0x0000000000000000
rip 0x0000000000000000 eflags 0x0000000000000000 cs base 0x0000000000000000 gs base 0x0000000000000000

Source
7 float gini_argentina = 40.7; // Un valor de prueba
8
9 // Se llama a la función en ensamblador
10 int resultado = procesar_indice_ensamblador(gini_argentina);
11 printf("El GINI original era: %f\n", gini_argentina);
12 printf("El resultado procesado por ASM es: %d\n", resultado);
13
14 return 0;
15
Stack
[0] from 0x0000000000000000 in main+40 at codigo_en_C_puro.c:12
Threads
[1] id 2265 name programa_en_C_p from 0x0000000000000000 in main+40 at codigo_en_C_puro.c:12
Variables
gini_argentina 40.7000000 resultado 42

>>> x/8w $rsp
0x7fffffff0d40: 0xffffde30 0x00007fff 0x4222cccd 0x0000002a
0x7fffffff0d50: 0xffffd0f0 0x00007fff 0xf7c2a1ca 0x00007fff
>>>

```

Figura 15: Carga del Resultado